

Introduction



Prof. K. Adishesha

Learning Outcomes

- ❑ Introduction to C++
- ❑ Tokens
- ❑ Data types
- ❑ Input and Output Operators
- ❑ Control statements
- ❑ Arrays
- ❑ Functions
- ❑ Structures
- ❑ OOPS Concepts



C++ Introduction

C++ Introduction

- ❑ C++ is an object-oriented programming language which allows code to be reused, lowering development costs.
- ❑ C++ was developed by Bjarne Stroustrup, as an extension to the C language.
- ❑ C++ gives programmers a high level of control over system resources and memory.



C++ Introduction

OOPs characteristics

- ❑ **Modularity:** Module is a logically self-contained unit that can be tested and executed independently.
- ❑ **Abstraction:** It represents the essential features of an entity without including explanations or any background details about it.
- ❑ **Data Encapsulation:** Wrapping of data and functions into a single unit is called data encapsulation.
- ❑ **Inheritance:** The process by which objects of one class acquires the properties of the objects of another class.
- ❑ **Polymorphism:** The ability for a message to be processed in more than one form.
- ❑ **Dynamic Binding:** Linking of a procedure call to the code to be executed when it is called.
- ❑ **Message Passing:** Passing message objects and invoking the function by the object by sending a message is known as message passing.



C++ Introduction

OOPs Benefits

- ❑ OOPs model the real world entity very well.
- ❑ Inheritance eliminates the redundancy (repetition) of code and hence supports code reusability.
- ❑ Data hiding helps to build secured programs.
- ❑ Multiple instances (objects) can be created.
- ❑ Work can be divided easily.
- ❑ OOPs can be easily upgraded from small to large systems.
- ❑ Complexity can be easily managed.
- ❑ Message passing concept helps the objects to communicate and share data..

C++ Introduction

OOPs Applications

- ❑ Object oriented databases.
- ❑ Hypermedia, expert text and hypertext.
- ❑ Artificial intelligence and expert systems.
- ❑ Decision support systems and office automation systems.
- ❑ Parallel programming and neural networks.
- ❑ CAD, CAM, CIM systems.
- ❑ Simulation and modeling.

C++ Introduction

Characteristics of C++:

❑ **Object-Oriented Programming:**

- It allows the programmer to design applications like a communication between object rather than on a structured sequence of code.

❑ **Portability:**

- We can compile the same C++ code in almost any type of computer & operating system without making any changes.

❑ **Modular Programming:**

- An application's body in C++ can be made up of several source code files that are compiled separately and then linked together saving time.

❑ **C Compatibility:**

- Any code written in C can easily be included in a C++ program without making any changes.

C++ Introduction

Characteristics of C++:

❑ **Speed:**

- The resulting code compilation is very efficient due to its duality as high-level and low-level language.

❑ **Flexibility:**

- It is highly flexible language and versatility.

❑ **Wide range of library functions:**

- It has huge library functions; it reduces the code development time and also reduces cost of software development.

❑ **System Software Development:**

- It can be used for developing System Software Viz., Operating system, Compilers, Editors and Database.



C++ Introduction

Translating a C++ program

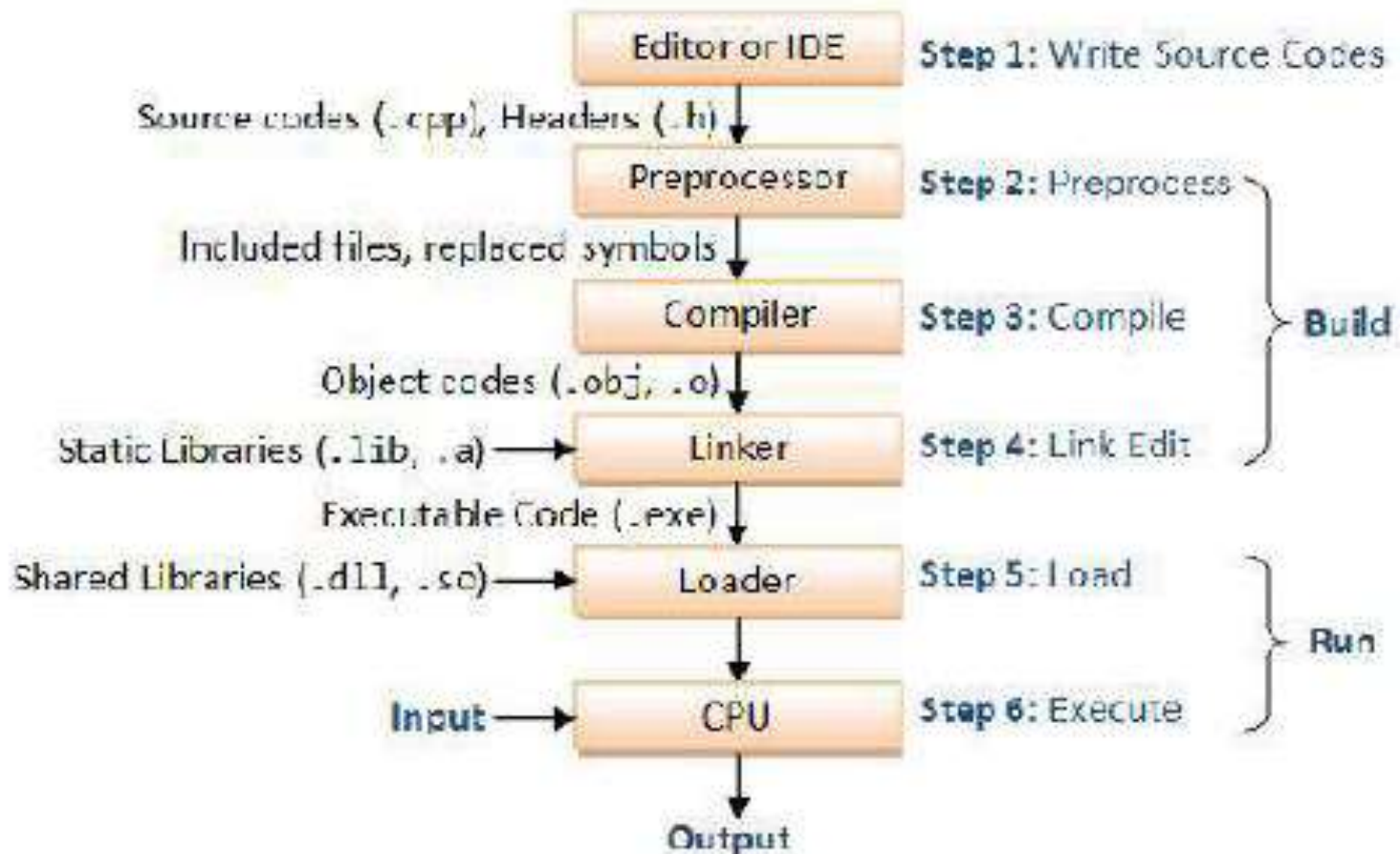
- ❑ Computers execute binary instructions.
- ❑ These binary instructions are known as machine instructions or machine code.
- ❑ The program creation process consists of the following steps:
 - Step 1** – Write the program in a computer language humans can read and understand (like C++),
 - Step 2** – Save the programs in text files. Programs can be a few lines long and reside in one file or can consist of many millions of lines of code and span thousands of files,
 - Step 3** – Run the source code files through a program called a compiler to generate object code for the target computer,
 - Step 4** – Run the object files through a program called a linker to produce an executable image.



C++ Introduction

Translating a C++ program

- The program execution process consists of the following steps:



C++ Introduction

General Structure of C++ Program

- ❑ Different programming languages have their own format of coding.
- ❑ The basic components of a C++ program are:

Simple C++ Program

```
// Hello World program  ← comment

#include <iostream.h>  ← Allows access to an I/O library

int main()
{  ← Starts definition of special function main()
    cout << "Hello World\n";  ← output (print) a string
    return 0;  ← Program returns a status code (0 means OK)
}
```



C++ Introduction

General Structure of C++ Program.

- ❑ Different programming languages have their own format of coding.
- ❑ The basic components of a C++program are:
 - **Comments or Documentation Section**
 - **Pre-processor Directives (Linker Section):**
 - **Definition**
 - **Global Declaration**
 - **main () function**
 - **Declarations**
 - **Statements**



C++ Introduction

C++ Character Set:

- The valid set of characters that a C++ language can recognizes includes the following.

Letters A-Z, a-z

Digits 0-9

Special Characters Space + - * / ^ \ () [] { } = != < > ' " \$, ; : % ! & ? _ # <= >= @

Formatting characters backspace, horizontal tab, vertical tab, form feed, and carriage return



C++ Introduction

Tokens:

- Smallest individual unit in a program is known as token.

1. Identifiers

2. Keywords

3. Literals

4. Operators

5. Punctuators / Delimiters



Identifiers

Identifiers is a name given to programming elements such as variables, functions, arrays, objects, classes, etc.,

- The following are some valid identifiers:

Student

Reg101

a1e2r3

_dos

- Rules to be followed while creating identifiers:
 - Identifiers are a sequence of characters which should begin with the alphabet either from A-Z (Uppercase) or a-z (lowercase) or _ (underscore).
 - C++ treats uppercase and lowercase characters differently
 - No Special character is allowed except underscore “_”.
 - Identifier should be single words i.e. blank spaces cannot be included in identifier.
 - Reserved Keywords should not be used as identifiers.
 - Identifiers should be of reasonable length.



Keywords

Keywords: are predefined word that gives special meaning to the complier.

alignas	continue	friend	register	true
alignof	decltype	goto	reinterpret_cast	try
asm	default	if	return	typedef
auto	delete	inline	short	typeid
bool	do	int	signed	typename
break	double	long	sizeof	union
case	dynamic_cast	mutable	static	unsigned
catch	else	namespace	static_assert	using
char	enum	new	static_cast	virtual
char16_t	explicit	noexcept	struct	void
char32_t	export	nullptr	switch	volatile
class	extern	operator	template	wchar_t
const	false	private	this	while
constexpr	float	protected	thread_local	
const_cast	for	public	throw	



Literals

Literals: A Literals/ constant are identifiers whose value does not change during program execution.

- ❑ Constants are sometimes referred to as literal.
- ❑ A constant or literal may be any one of the following:
 - **Integer Constant**
 - **Floating Constant**
 - **Character Constant**
 - **String Constant**



Literals

Integer Constant:

- ❑ An integer constant is a whole number, which can be either positive or negative.
- ❑ They do not have fractional part or exponents.
- ❑ We can specify integer constants in:
 - **Decimal Integer Constant**
 - ❑ `int a = 120; //Decimal Constant`
 - **Octal Integer Constant**
 - ❑ `int b = 0374; //Octal Constant`
 - **Hexadecimal Integer Constant**
 - ❑ `int c = -0XABF; //Hexadecimal Constant`
 - **Unsigned Constant**
 - ❑ `unsigned d = 328u; //Unsigned value`



Literals

Floating Point Constant:

- ❑ Floating point constants are also called as “real constants”.
- ❑ These values contain decimal points (.) and can contain exponents.
- ❑ They are used to represent values that will have a fractional part and can be represented in two forms (i.e. fractional form and exponent form)
- ❑ We can specify Floating Point constants in:
 - **float a=23.46** // equal to $23.46 \times 10^0 = 23.46 \times 1 = 23.46$
 - **float b=26.126**



Literals

Character constants: are specified as single character enclosed in pair of single quotation marks.

- For example `char ch = 'P';`
- There are certain characters used in C++ which represents character constants called as escape sequence which starts with a back slash (\) followed by a character.

Escape Sequence	Meaning	Escape Sequence	Meaning
\'	Single Quote	\"	Double Quote
\?	Question Mark	\\	Back Slash
\0	Null Character	\a	Audible Bell
\b	Backspace	\f	New Page
\n	New Line	\r	Carriage Return
\t	Horizontal Tab	\v	Vertical Tab
\nnn	Arbitrary octal value	\xnn	Arbitrary Hexa Value



Literals

String Constants:

- ❑ A string constant consists of zero or more character enclosed by double quotation marks (“ “).
- ❑ Multiple character constants are called string constants and they are treated as an array of char.
- ❑ By default compiler adds a special character called the “Null Character” (**\0**) at the end of the string to mark the end of the string.
- ❑ Example:
 - **char str[25] = “Hello Adishesha” ;**
 - This is actually represented as char str[25] = “Hello Adishesha\0” in the memory



C++ Operators

□ C++ Operators:

- Operators are used to perform operations on variables and values.
- Example: `int x = 100 + 50;`

□ C++ Operators Types:

- C++ divides the operators into the following groups.
 - **Arithmetic operators**
 - **Assignment operators**
 - **Comparison operators**
 - **Logical operators**
 - **Bitwise operators**



Arithmetic Operators

- Arithmetic operators are used to perform common mathematical operations.

Operator	Description	Example(a=10, b=20)
+	Adds two operand	$a + b = 30$
-	Subtracts second operand from the first	$a - b = -10$
*	Multiply both operand	$a * b = 200$
/	Divide numerator by denominators	$b / a = 2$
%	Modulus operators and remainder of after an integer division	$b \% a = 0$



Assignment Operators

- Assignment operators are used to assign values to variables.

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3



Relational Operator

- ❑ Comparison operators are used to compare two values.
- ❑ The return value of a comparison is either true (1) or false (0).

Operator	Description	Example (a=10, b=5)
<	Checks if the value of left operand is less than the value of right operand	a < b returns false(0)
<=	Checks if the value of left operand is less than or equal to the value of right operand	a <= b returns false(0)
>	Checks if the value of left operand is greater than the value of right operand	a > b returns true(1)
>=	Checks if the value of left operand is greater than or equal to the value of right operand	a >= b returns false(0)
==	Checks if the value of two operands is equal or not	a == b returns false(0)
!=	Checks if the value of two operands is equal or not	a != b returns true(1)



Logical Operators

- Logical operators are used to determine the logic between variables or values.

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	<code>x < 5 && x < 10</code>
	Logical or	Returns true if one of the statements is true	<code>x < 5 x < 4</code>
!	Logical not	Reverse the result, returns false if the result is true	<code>!(x < 5 && x < 10)</code>



Bitwise Operators

- A Bitwise operators are used in bit level programming.

Operators	Meaning of operators
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
~	Bitwise complement



C++ Operators

- ❑ Operators may also be classified on the number of operands they act on either:
 - **Unary Operators**
 - Example: `a++, a+1`
 - **Binary Operators**
 - Example: `x = x - 10;`
 - **Ternary Operators**
 - Example: `x = (a>b) ? a:b;`

Unary	Binary	Ternary
Unary operators Operate on only one operand.	Binary operator Operates on two operands.	Ternary operator Operates on three operands.
Example: <code>++, --</code>	<code>+, -, *, /, %, &&, </code>	<code>?:</code>



C++ Operators

Special Operator :

- ❑ An expression is a combination of opcode and operand.
- ❑ Some special operators used in C++ programming are:

Operators	Meaning of operators
sizeof()	It is a unary operator which is used in finding the size of the data type. Example: sizeof(a)
, (comma)	Comma operators are used to linking related expressions together. Example: int a=10, b=5
. (dot) and -> (arrow)	Member Operator used to reference individual members of classes, structure and unions.
cast	Casting Operator convert one data type to another.
&	Address Operator & returns the address of the variable.
*	Pointer Operator * is pointer to a variable.



Punctuators

- ❑ Punctuators in C++ have syntactic and semantic meaning to the compiler.

Punctuator	Name	Function
!	Exclamation	Used along with “=” to indicate “not equal to”
%	Percentage	Used along with format specifiers
&	Ampersand	Used to represent address location or bitwise operation
;	Semicolon	Used to represent statement terminator.
[]	Brackets	Used to array subscripts
()	Parenthesis	Used to represent function calls and function parameters.
{ }	Braces	Used to represent start and end of a block of code.
#	Ash sign	Used to represent preprocessor directives.
\	Back slash	Used to represent escape sequence
:	Colon	Used to represent labeled statement
=	Equal to	Used to represent an assigning operator.



Data Types

Data Types:

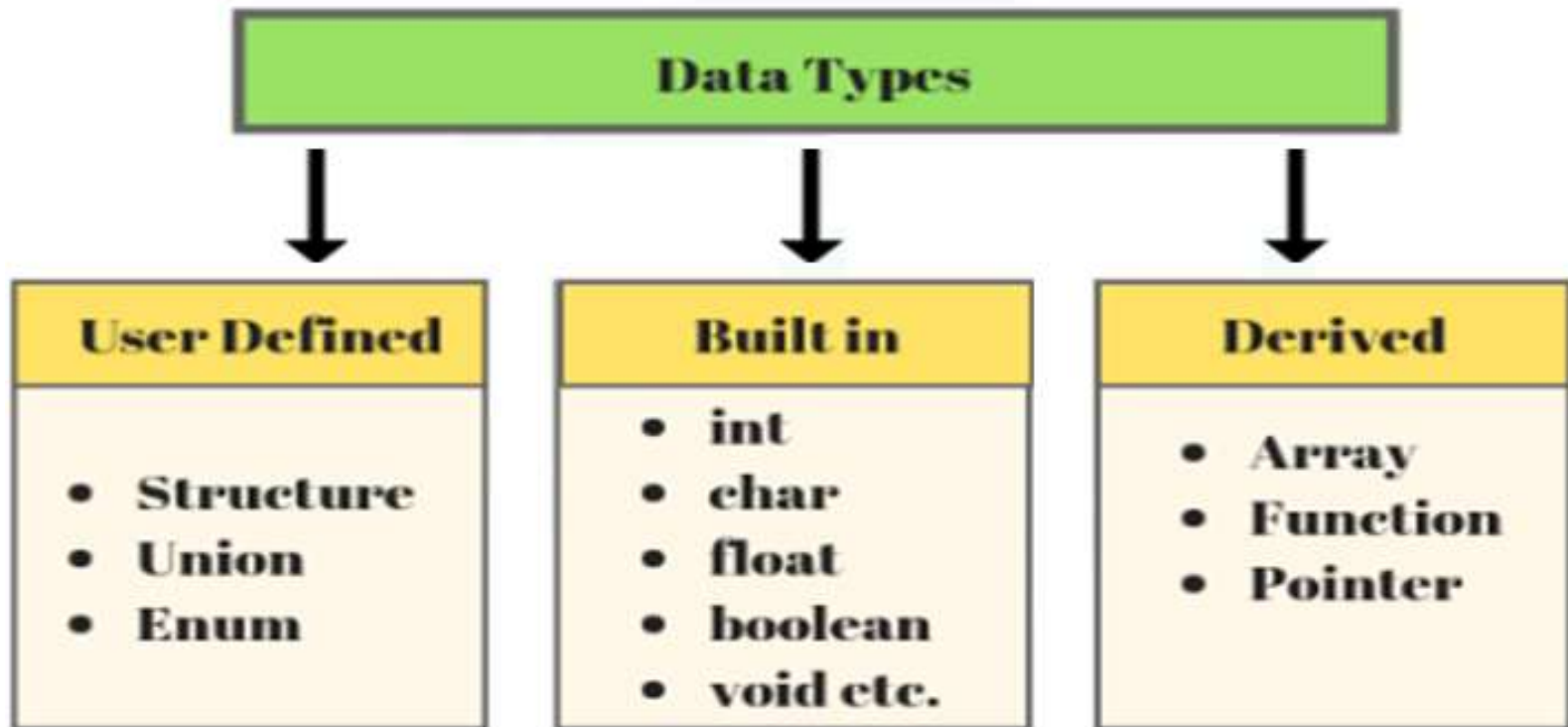
- ❑ Data Types can be defined as the set of values, which can be stored in a variable along with the operations that can be performed on those values.
- ❑ C++ defines several types of data and each type has unique characteristics.
- ❑ C++ data types can be classified as:
 - 1. The fundamental data type(built-in data)**
 - 2. Derived Data type**
 - 3. User-defined data type**



Data Types

Data Types:

- ❑ C++ defines several types of data and each type has unique characteristics.
- ❑ C++ data types can be classified as:



Data Types

Basic Data Types:

- The data type specifies the size and type of information the variable will store.

Type	Size in bytes	Range	Examples
int	2	-32768 to 32767	8, 100, -39
char	1	-128 to 127	'd', '6', '#'
float	4	3.4×10^{-38} to $3.4 \times 10^{38} - 1$	45.345, 0.134, 3.142
double	8	1.7×10^{-308} to $1.7 \times 10^{308} - 1$	3.1415677888888888



Type Conversion

Converting an expression of a given type into another type is known as **typecasting or type conversion**.

- Type conversions are of two types, they are:
 - **Implicit Conversion:** They are automatically performed when a value is copied to a compatible type.
 - Example: `short a = 2000;`
 `int b;`
 `b = a;`
 - **Explicit Conversion:** Many conversions, especially those that imply a different interpretation of the value, require an explicit conversion.
 - Example: `short a = 2000;`
 `int b;`
 `b = (int) a; //c-like cast notation`



Input & Output Operators

Input & Output Operators

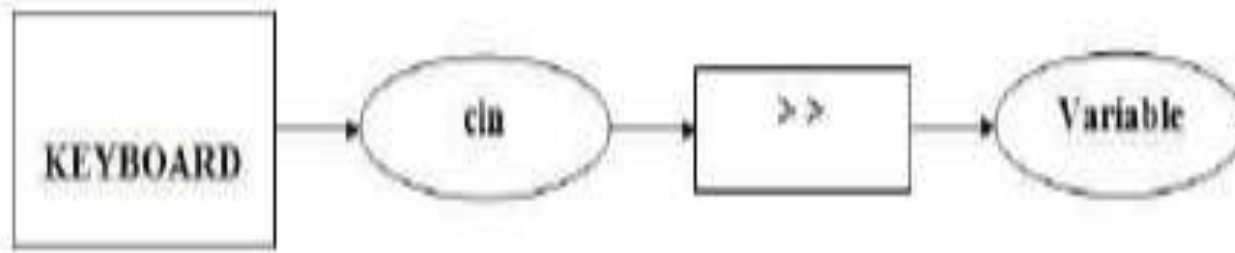
- ❑ The input output operations are done using library functions **cin** and **cout** objects of the class **iostream**.
- ❑ Using the standard input and output library, we will be able to interact with the user by printing message on the screen and getting the user's input from the keyboard.
 - A stream is an object where a program can either insert/extract characters to/from it.
 - The standard C++ library includes the header file `iostream`, where the standard input and output stream objects are declared.



Input & Output Operators

Input Operators:

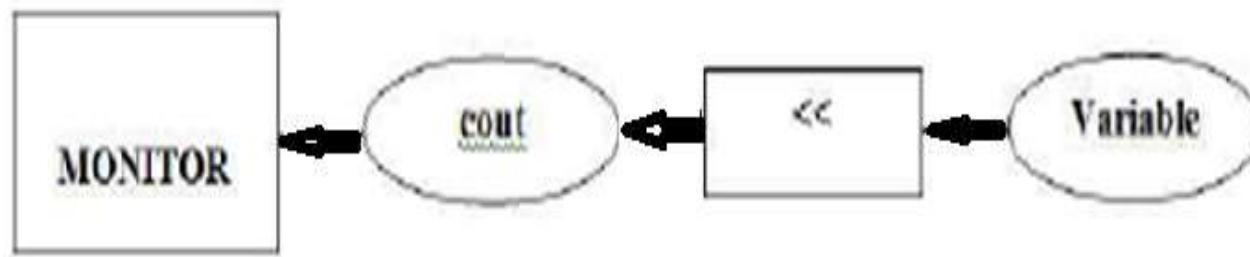
- ❑ Input Operator “>>”: The standard input device is usually the keyboard.
- ❑ Input in C++ is done by using the “stream extraction” (>>) on the cin stream.
- ❑ “cin” stands for “console input”.
- ❑ Example: **int age;**
cin>>age;



Input & Output Operators

Output Operator:

- ❑ Output Operator “<<”: The standard output device is the screen (Monitor).
- ❑ Outputting in C++ is done by using the object followed by the “stream insertion” (<<).
- ❑ “cout” stands for console output
- ❑ Example: **cout<<“sum”; //prints sum**
cout<<sum; //prints the content of the variable sum;



Input & Output Operators

Cascading of I/O Operators:

- ❑ If a program requires more than one input variable then it is possible to input these variables in a single **cin** statement using multiple stream extraction “>>” operators.
- ❑ Example: **cout<<“Enter the two number”;**
cin>>a>>b;
- ❑ If a program requires more than one output result then this can be done using a single **cout** statement with multiple stream insertion “<<” operators.
- ❑ This is called cascading of input & output operators.
- ❑ Example: **cout<<“Entered the two number”<<a<<b<<endl;**
cout<<“The sum of two number is”<<sum<<endl;



Manipulator

Formatted Output (Manipulators): :

- ❑ Manipulators are the operators used with the insertion operator “<<” to format the data display.
- ❑ To use manipulator it is must to include header file <iomanip.h>
 - **endl**
 - **setw()**
- ❑ Example:
- ❑ **endl manipulator:** causes a line feed to be inserted. It has same effect as using new line character “\n”.
cout<<”Entered the two number”<<a<<b<<endl;
- ❑ The **setw()** manipulator sets the width of the field assign for the output.
cout<<setw(6)<<”R” ;
Output: _ _ _ _ _ R



Manipulator

Formatted Output (Manipulators):

- ❑ Manipulators are the operators used with the insertion operator “<<” to format the data display.
- ❑ Program: To find the sum of two numbers:

```
#include<iostream.h>
#include<iomanip.h>
void main( )
{   int a, b, add;
    clrscr( );
    cout<<"Enter the two numbers"<<endl;
    cin>>a>>b;
    add = a + b;
    cout<<"The sum of two number is"<<setw(6)<<sum<<endl;
    getch();
}
```



Control Statements

Introduction:

- ❑ Control statements are statements that alter the sequence of flow of instructions.
- ❑ The order in which statements are executed in a program is called flow of control.
- ❑ Types of control statements: C++ supports two basic control statements.
 - **Selection statements**
 - **Iteration statements**



Control Statements

Selection Statements:

- ❑ This statement allows us to select a statement or set of statements for execution based on some condition.
- ❑ It is also known as conditional statement.
- ❑ This structure helps the programmer to take appropriate decision.
- ❑ The different selection statements:
 - **if statement**
 - **if – else statement**
 - **Nested – if statement**
 - **switch statement**



Selection Statements

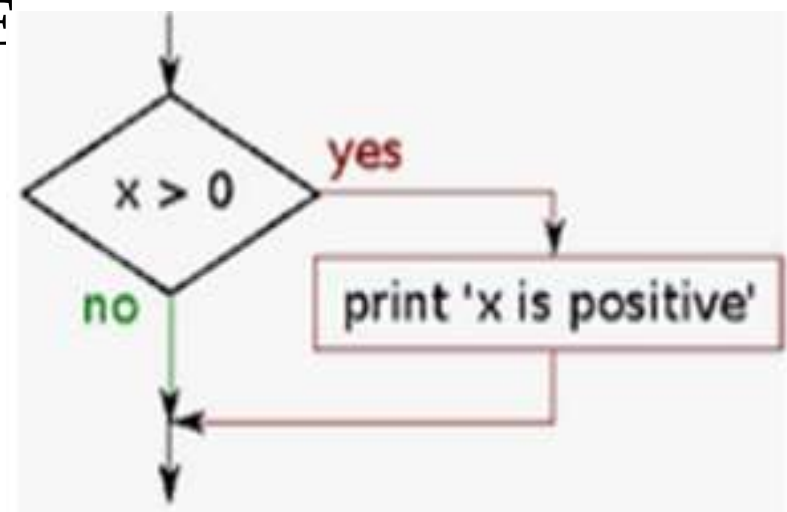
if statement:

- ❑ This is the simplest form of Selection statement.
- ❑ This statement is also called as one-way branching.
- ❑ This statement is used to decide whether a statement or set of statements should be executed or not.
- ❑ The decision is based on a condition which can be evaluated to TRUE or FALSE
- ❑ Syntax:

if (Test Condition) // is true

Statement 1;

Statement 2;



Selection Statements

if – else statement:

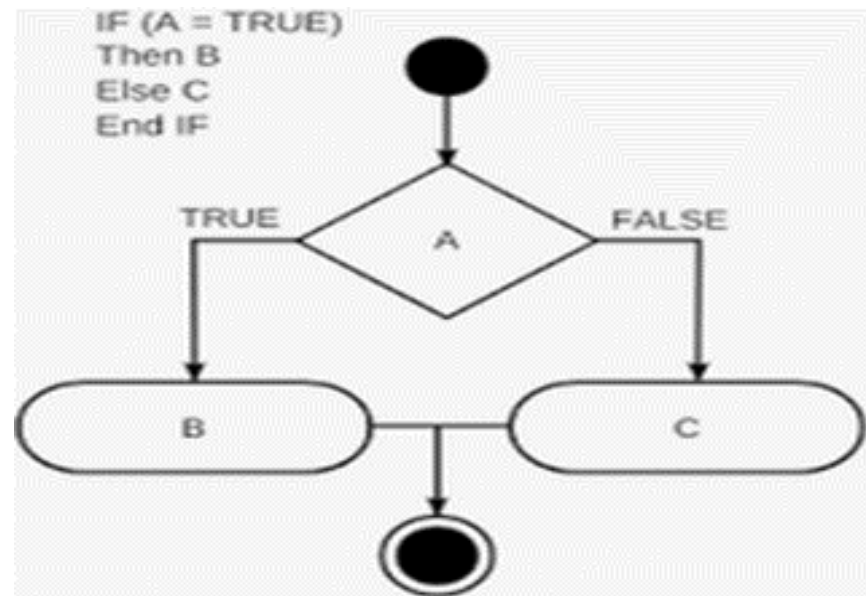
- ❑ This statement is also called as two-way branching.
- ❑ This structure helps to select one set of statements to be executed from two sets.
- ❑ Syntax of if – else statement is:

if (Test Condition)

Statement 1;

else

Statement 2;



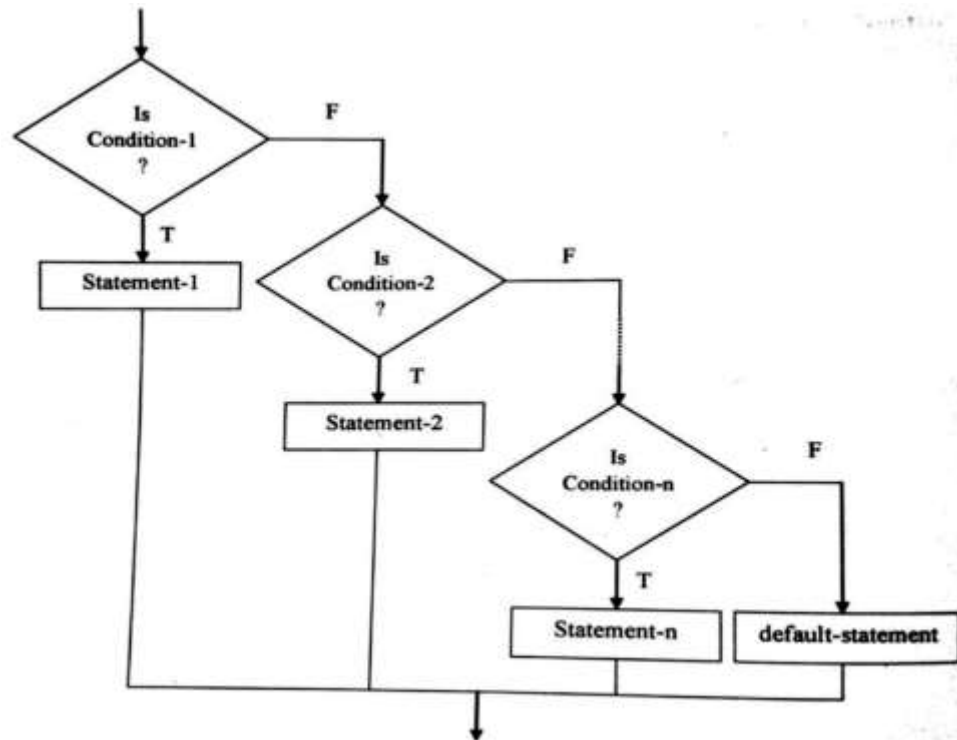
Selection Statements

Nested if statement :

- If the statement of an if statement is another if statement then such an if statement is called as Nested-if Statement.
- The general form of if – else – if statement is:.

if (Test Condition 1)
 Statement 1;
else if (Test Condition 2)
 Statement 2;
else

else if(test Condition N)
 Statement N;
else Default Statement;



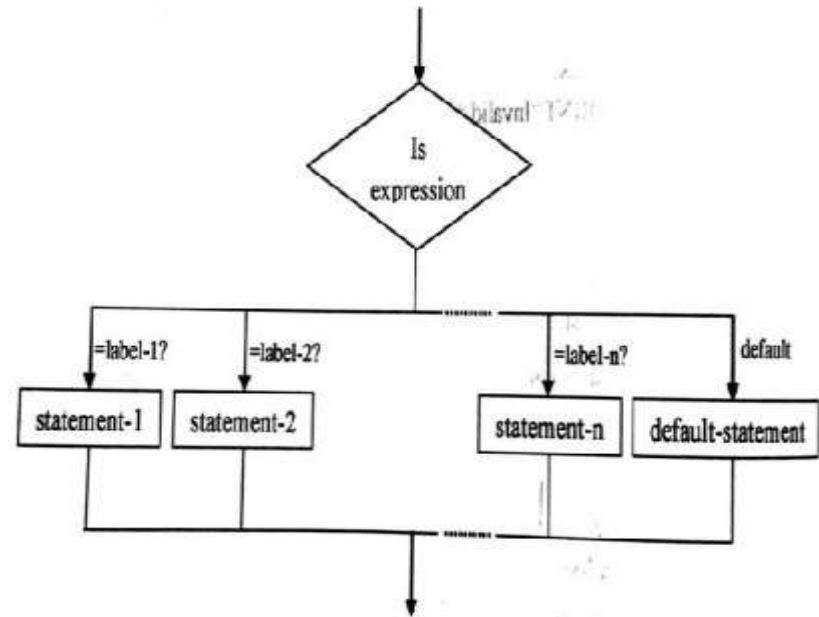
Selection Statements

Switch Statement :

- ❑ C++ has built in multiple-branch selection statement
- ❑ If there are more than two alternatives to be selected, multiple selection construct is used.
- ❑ The general form of Switch statement is:

Switch (Expression)

```
{ Case Label-1: Statement 1;  
Break;  
Case Label-2: Statement 1;  
Break;  
.....  
Case Label-N: Statement N;  
Break;  
Default : Default- Statement;  
}
```



Iteration statements

Iterative Constructs or Looping:

- ❑ The process of repeated execution of a sequence of statements until some condition is satisfied is called as iteration or loop.
- ❑ Iterative statements are also called as repetitive statement or looping statements.
- ❑ There are three types of looping structures in C++:
 - **while loop**
 - **do while loop**
 - **for loop**



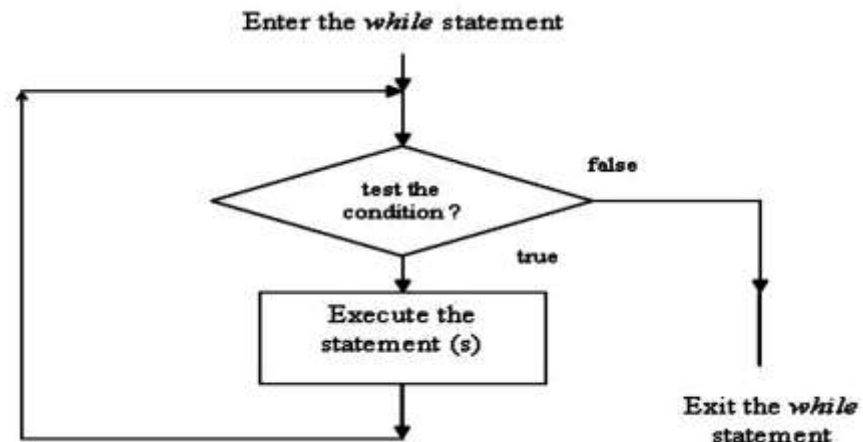
Iteration statements

while loop:

- ❑ This is a pre-tested loop structure.
- ❑ This structure checks the condition at the beginning of the structure.
- ❑ The set of statements are executed again and again until the condition is true.
- ❑ The general form of while structure is

while (Test Condition)

```
{  Statement 1  
  Statement 2  
  .....  
  Statement N  
}  //End of While
```



Iteration statements

do-while loop:

- ❑ This is a post-tested loop structure.
- ❑ This structure checks the condition at the end of the structure.
- ❑ The set of statements are executed again and again until the condition is true.
- ❑ The general form of while structure is

do

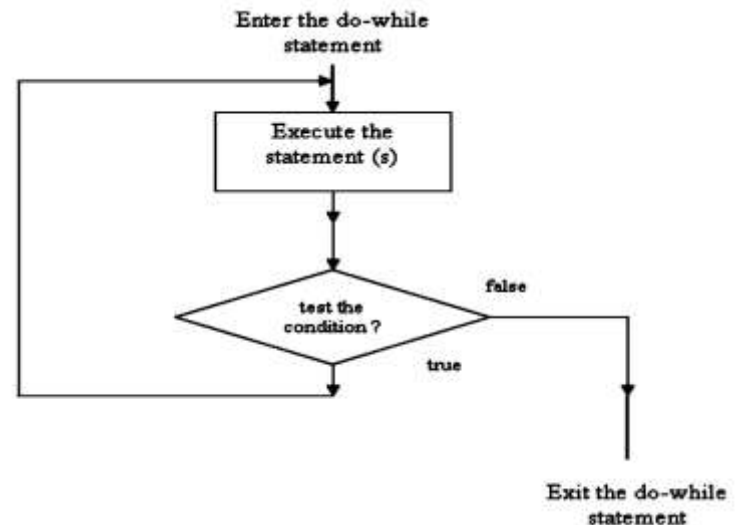
{ Statement 1

Statement 2

.....

Statement N

} //while (Test Condition);



Iteration statements

for loop:

- ❑ This structure is the fixed execution structure.
- ❑ Usually used when we know in advance exactly how many times a set of statements to be executed repeatedly.
- ❑ This structure can be used as increment looping or decrement looping structure.
- ❑ The general form of for structure is as follows:

```
for ( Expression 1; Expression 2; Expression 3 )  
{    Statement 1;  
        Statement 2;  
        Statement N;  
}
```



Transfer Control

Jump statements :

- ❑ Jump or Transfer control within looping are used to
 - Terminate the execution of loop.
 - Exiting a loop.
 - Half way through to skip the loop.
- ❑ The general form of Jump or Transfer control are:
 - **break statement**
 - **exit() statement**
 - **continue statement**
 - **goto statement**



Transfer Control

break statement:

- ❑ The break statement has two uses
 - To terminate a case in the switch statement.
 - To force immediate termination of a loop like while, do-while and for, by passing the normal loop conditional test.
- ❑ When the break statement is encountered the loop is immediately terminated and program control resumes at the next statement.
- ❑ The general form of break statement is:

```
for (n=0; n<100; n++)  
{  
    cout<<n;  
    if(n==10)  
        break;  
}
```

Transfer Control

exit() function:

- ❑ This function causes immediate termination of the entire program, forcing a return to the operating system.
- ❑ In effect, exit() function acts as if it were breaking out of the entire program.
- ❑ The general form of exit() statement is:

```
for (n=0; n<100; n++)  
{      cout<<n;  
        if(n==10)  
        exit(0);  
}
```



Transfer Control

continue statement:

- ❑ The continue statement causes the program to skip the rest of the loop in the current iteration, causing it to jump to start of the following iteration.
- ❑ Instead of forcing termination, **continue** forces the next iteration of the loop to take place.
- ❑ The general form of continue statement is:

```
for (n=10; n>0; n--)  
{  
    if(n==10)  
        continue;  
    cout<<n;  
}
```



Transfer Control

goto statement:

- ❑ The goto allows to makes an absolute jump to another point in the program.
- ❑ This statement execution causes an unconditional transfer of control from one statement to the other statement.
- ❑ A label is made of a valid identifier followed by a colon (:).
- ❑ The general form of goto statement is:

```
void main( )  
{      int n=10;  
      loop: cout<<"\t"<<n;  
          n--;  
          if(n>0) goto loop;  
          cout<<"End of loop";  
          getch( ); }
```



Arrays

Introduction:

- ❑ An array is collection of elements where all the elements are same data type under the same name.
- ❑ The elements are numbered as 0, 1, 2....n-1.
- ❑ These numbers called as indices or subscripts.
- ❑ These numbers are used to locate the positions of elements within the array.
- ❑ If a is the name of the array, the elements can be directly accessed as

$a[0], a[1], a[2], \dots, a[n-1]$.



Arrays

Introduction:

- ❑ An array is collection of elements where all the elements are same data type under the same name.
- ❑ The method of numbering the i th element with index $i-1$ is called as zero- based indexing.
- ❑ **Types of Arrays:**
 - **One-dimensional array**
 - ❑ Each element is accessed using an array with one subscript
 - **Two-dimensional array**
 - ❑ Each element is accessed using 2-subscripts in an array
 - **Multi-dimensional array**
 - ❑ A multi-dimensional array is an array of n -dimensions



Functions

Introduction:

- A function is a named group of statements developed to solve a sub-problem and returns a value to other functions when it is called.
- **Types of functions:**
 - **Library functions**
 - A standard library is a collection of pre-defined functions and other programming elements, which are accessed through header files
 - **User-defined functions**
 - We can create our own functions or sub-programs to solve our problem. Such functions are normally referred to as user-defined functions



Functions

User-defined functions:

- ❑ User-defined function is a function defined by the user to solve his/her problem.
- ❑ The purpose of using a function is to make the program design process easy, understandable and thereby avoiding ambiguity.
- ❑ **Types of functions:**
 - **Function with no arguments and no return values.**
 - **Function with arguments and with no return values.**
 - **Function with no arguments and with return values.**
 - **Function with arguments and with return values.**
 - **Recursive function.**



Structures

Introduction:

- ❑ A structure is a collection of various data elements under one name.
- ❑ The variables in a structure can be of same or different types.
- ❑ The data items in a structure are called the members of the structure.
- ❑ **Syntax:** **struct structure-name**

{

datatype member-name-1;

datatype member-name-2;

.....

datatype member-name-n;

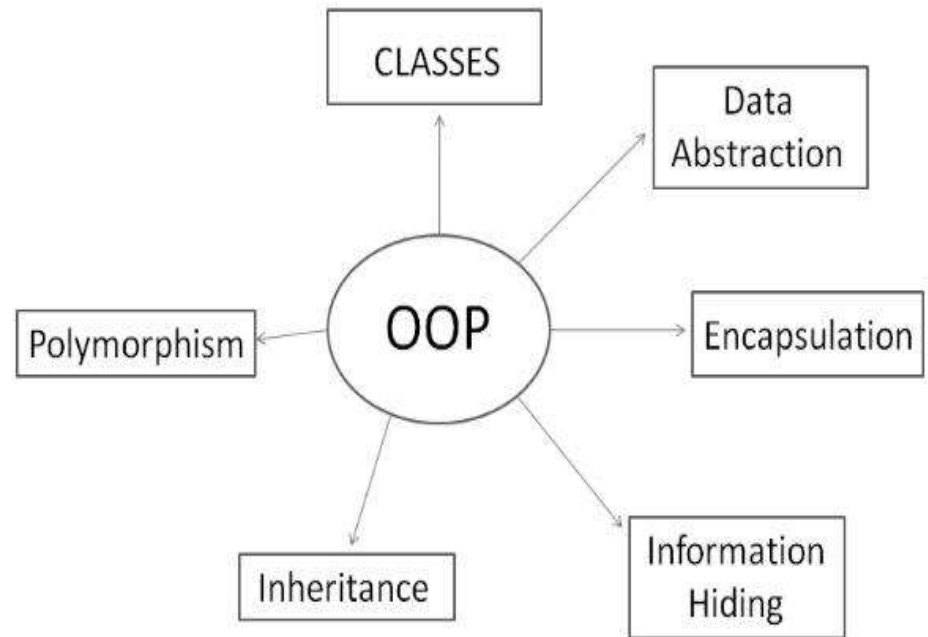
};



Basic Concepts of OOP's

□ The following are the major characteristics of OOP's:

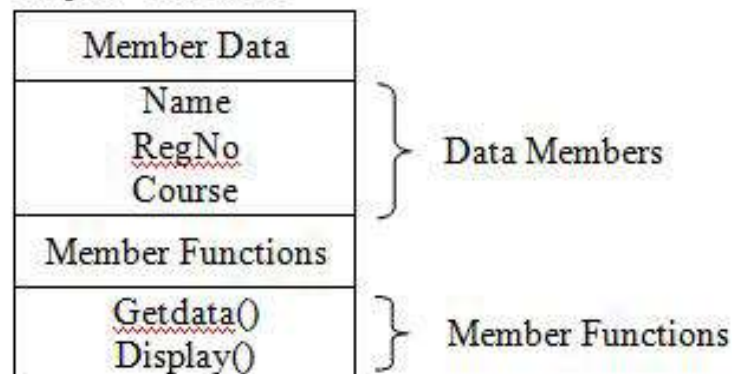
- Class
- Objects
- Data abstraction
- Data encapsulation
- Inheritance
- Overloading
- Polymorphism
- Dynamic Binding
- Message Passing



Objects

- ❑ Objects are basic building blocks for designing programs.
- ❑ An object is a collection of data members and associated member functions.
- ❑ An object may represent a person, place or a table of data.
- ❑ Each object is identified by a unique name. Each object must be a member of a particular class.
- ❑ Example: Adi, Sunny, Prajwal are the objects of class PUC.

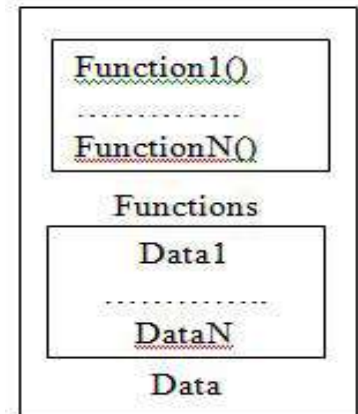
Object - Student



Class

- ❑ A class is a collection of objects that have identical properties, common behavior and shared relationship.
- ❑ A class binds the data and its related functions together.
- ❑ A class is a user-defined data type that we can use in a program.
- ❑ To create a class, use the class keyword.
- ❑ Example:

```
class MyClass
{
    // The class body
};
```



- Where **class** keyword is used to create a class called **MyClass**



Class in C++

Definition and Declaration of Classes

- ❑ A class definition is a process of naming a class and data variables, and interface operation of the class.
- ❑ The variables declared inside a class are known as data members.
- ❑ The functions declared inside a class are known as member functions.
- ❑ A class declaration specifies the representation of objects of the class and set of operations that can be applied to such objects.



Class in C++

Definition and Declaration of Classes

- ❑ Class body is enclosed in a pair of flower brackets. Class body contains the declaration of its members (data and functions).
- ❑ The functions declared inside a class are known as member functions.
- ❑ The general syntax of the class declaration is:

```
class User_Defined_Name  
{  
    private :  
        Data Member;  
        Member functions;  
};
```



Access Specifiers

Access Specifiers

- ❑ Access specifiers define how the members (attributes and function) of a class can be accessed.
- ❑ Every data member of a class is specified by three levels of access protection for hiding data and function members internal to the class.
- ❑ They help in controlling the access of the data members.
- ❑ Different access specifiers are:
 - **private**
 - **public**
 - **protected**



Access Specifiers

private:

- ❑ private access means a member data can only be accessed by the class member function or friend function.
- ❑ The data members or member functions declared private cannot be accessed from outside the class.
- ❑ The objects of the class can access the private members only through the public member functions of the class.
- ❑ By default data members in a class are private.
- ❑ Example:

private:

int x;

float y;



Access Specifiers

protected:

- ❑ The members which are declared using protected can be accessed only by the member functions, friend of the class and also the member functions derived from this class.
- ❑ The members cannot be accessed from outside the class.
- ❑ The protected access specifier is similar to private access specifiers.
- ❑ Example:

protected:

int x;

float y;



Access Specifiers

public:

- ❑ public access means that member can be accessed any function inside or outside the class.
- ❑ Some of the public functions of a class provide interface for accessing the private and protected members of the class.
- ❑ Example:

```
class MyClass
{
    public: // Public access specifier
        int x; // Public attribute
    private: // Private access specifier
        int y; // Private attribute
};
```



Member Function

Member Function:

- ❑ Member functions are functions that are included within a class (Member functions are also called Methods).
- ❑ Member functions can be defined in two places.
 - **Inside class definition**
 - **Outside class definition**



Member Function

Inside class definition:

- ❑ To define member function inside a class the function declaration within the class is replaced by actual function definition inside the class.
- ❑ Only small functions are defined inside class definition.
- ❑ Example:

```
class rectangle
{
int length, breadth, area;
public:
void get_data( )
{
cout<< " Enter the values for Length and
Breadth";
cin>>length>>breadth;
}
```

```
void compute( )
{
area = length * breadth;
}
void display( )
{
cout<<" The area of rectangle is"<<area;
}
};
```



Member Function

Outside class definition:

- ❑ To define member function outside the class, the class name must be linked with the name of member function.
- ❑ Scope resolution operator (::) is used to define the member function outside the class.
- ❑ Syntax of a member function defined outside the class is:

```
return_type class_name :: member_function_name( arg1, ..., argnN)  
{  
    function body;  
}
```



Member Function

Program to use member functions inside and outside class definition:

```
#include<iostream.h>
class item
{
private:
int numbers;
float cost;
public:
void getdata(int a, float b);
void putdata( )
{
cout<<"Number:
"<<number<<endl;
cout<<"Cost:"<<cost<<endl;
}
```

```
};
void item : : getdata(int a, float b)
{
number = a;
cost = b;
}

int main( )
{
item x;
x. getdata( 250, 10.5);
x.putdata( );
return 0;
}
```



Defining object of a class

Object of a class:

- ❑ An object is a real world element which is identifiable entity with some characteristics (attributes) and behavior (functions).
- ❑ An object is an instance of a class.
- ❑ An object is normally defined in the main () function.
- ❑ The syntax for defining objects of a class as follows:

```
class Class_Name  
{  
    private : //Members  
    public : //Members  
};
```

```
class Class_Name Object_name1, Object_name2,.....;
```

- where class keyword is optional.



Accessing member of the class

Dot operator (.):

- ❑ The public data members of objects of a class can be accessed using direct member access operator (.).
- ❑ Private and protected members of the class can be accessed only through the member functions of the class.
- ❑ No functions outside a class can include statements to access data directly.
- ❑ The syntax of accessing member (data and functions) of a class is:
 - a) Syntax for accessing a data member of the class:
Object_Name . data_member;
 - b) Syntax for accessing a member function of the class:
Object_Name . member_function(arguments);



Array of Objects

- ❑ An array having class type elements is known as array of objects.
- ❑ An array of objects is declared after definition and is defined in the same way as any other array.

- ❑ Example:

```
class employee
```

```
{
```

```
private:
```

```
    char name[10];
```

```
    int age;
```

```
public:
```

```
void readdata( );
```

```
void displaydata( );
```

```
};
```

```
employee Lecturer[15];
```

```
employee Admin[2];
```

- In the above example, the class employee has two array of objects contains 15 Lecturers and 2 Admin as objects.



Objects as function arguments

Function arguments

- ❑ A function can receive an object as a function argument.
- ❑ This is similar to any other data being sent as function argument.
- ❑ An object can be passed to a function in two ways:
 - **Pass by value:** Copy of entire object is passed to function.
 - **Pass by reference:** Only address of the object is transferred to the function.



Objects as function arguments

Pass by Value

- ❑ In pass by value, copy of object is passed to the function.
- ❑ The function creates its own copy of the object and uses it.
- ❑ Therefore changes made to the object inside the function do not affect the original object.
- ❑ Syntax:

```
void functionName(obj1, obj2)
{ // code to be executed
}

Void main( )
{
    functionName(val1, val2)
}
```



Objects as function arguments

Pass by Reference:

- ❑ In pass by reference, when an address of an object is passed to the function, the function directly works on the original object used in function call.
- ❑ This means changes made to the object inside the function will reflect in the original object, because the function is making changes in the original object itself.
- ❑ Pass by reference is more efficient, since it requires only passing the address of the object and not the entire object.
- ❑ Syntax:

void functionName(&obj1, &obj2)



Structure and Classes

Difference between Structure and Classes

Structure	Classes
A structure is defined with the struct keyword	A class is defined with the class keyword
All the member of a structure are public by default	All the members of a class are private by default
Structure cannot be inherit	Class can be inherit
A structure contains only data member	A class contain both data member and member functions
There is no data hiding features	Classes having data hiding features by using access specifiers(public, private, protected).



IMPORTANT QUESTIONS

One Mark questions:

1. What is a Class, Objects, Data Member, Member Functions, Scope Resolution Operator, and Array of objects?
2. Mention the access specifiers used with a class?

Five Mark questions:

1. Explain class definitions and class declaration with syntax and example.
2. Explain Member function. a. Inside class definition b. Outside class definition
3. Explain the array of objects?

